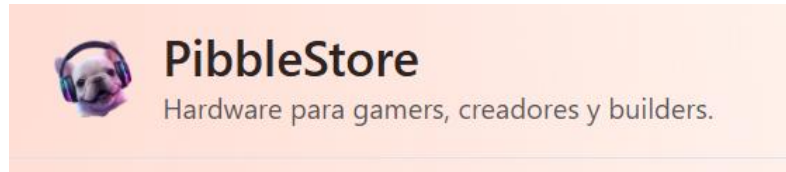


PibbleStore

PibbleStore es una tienda en línea de demostración enfocada en hardware para gamers, creadores y builders.

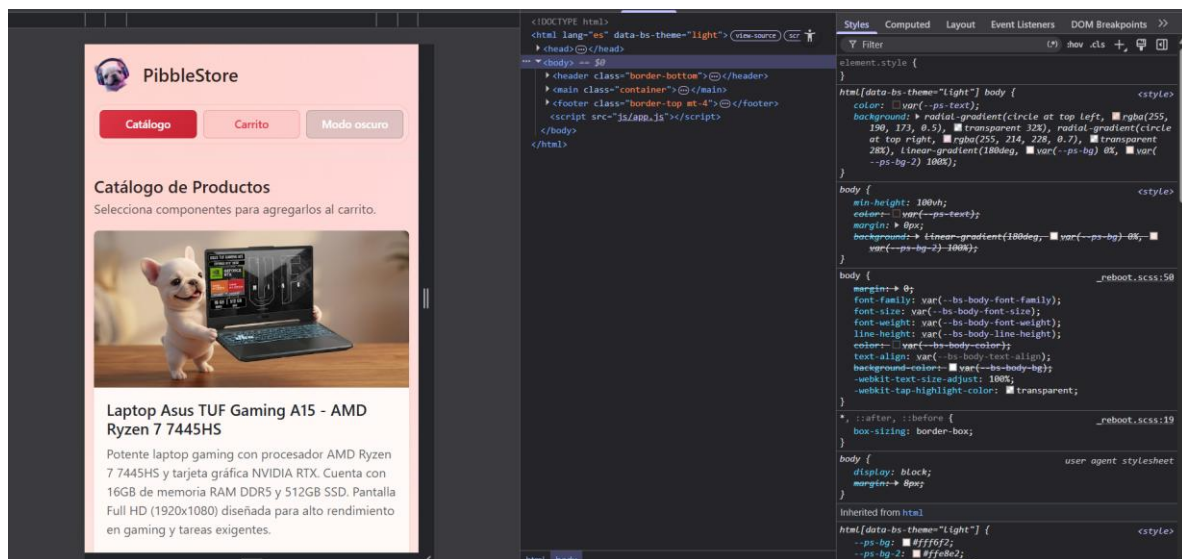
El proyecto está construido como una aplicación web estática con **HTML, CSS y JavaScript**, usando **Bootstrap 5.3** para la interfaz visual y **localStorage** para persistir el carrito de compras y la preferencia de tema.



Objetivo del proyecto

El objetivo principal fue desarrollar una experiencia de e-commerce sencilla pero funcional, con estas características:

- Mostrar un catálogo de productos de hardware.
- Permitir agregar productos al carrito.
- Guardar el carrito en el navegador del usuario.
- Alternar entre modo claro y modo oscuro.
- Mantener una interfaz responsiva y consistente en distintos tamaños de pantalla.



Tecnologías utilizadas

- **HTML5** para la estructura semántica del sitio.
- **CSS3** para personalización visual, estados interactivos y adaptaciones responsivas.
- **JavaScript** para la lógica dinámica de la aplicación.

- **Bootstrap 5.3** para la maquetación, componentes base y sistema de utilidades.
- **Bootstrap Icons** para los iconos usados en acciones como eliminar productos.
- **localStorage** para persistir datos en el navegador.

Estructura del proyecto

```
PW_P3/
├── INDEX.HTML
├── CSS/
│   └── STYLES.CSS
├── JS/
│   └── APP.JS
└── MEDIA/
    ├── LOGO.PNG
    ├── 1.PNG
    ├── 2.PNG
    ├── 3.PNG
    ├── 4.PNG
    ├── 5.PNG
    └── 6.PNG
```

Descripción de archivos

- **INDEX.HTML**: contiene la estructura principal de la página.
- **CSS/STYLES.CSS**: centraliza la apariencia personalizada del sitio.
- **JS/APP.JS**: controla el catálogo, carrito, tema visual y persistencia.
- **MEDIA/**: almacena el logo y las imágenes de los productos.

Estructura HTML

La página está organizada de forma clara en cuatro bloques principales:

1. HEADER

El encabezado incluye:

- El logo de la tienda.
- El nombre del proyecto.
- Una breve descripción.
- Un menú de navegación con tres botones:
 - CATÁLOGO
 - CARRITO
 - MODO OSCURO / MODO CLARO

Este bloque funciona como zona de control principal para cambiar de vista y alternar el tema visual.

2. MAIN

El contenido principal se divide en dos secciones:

- **Catálogo** (`#CATALOGO-VIEW`)
- **Carrito de compras** (`#CARRITO-VIEW`)

Ambas vistas existen al mismo tiempo en el HTML, pero JavaScript las oculta o muestra según la interacción del usuario.

Catálogo

En la vista del catálogo se encuentra:

- Un título.
- Una breve descripción.
- El contenedor `#PRODUCTS-CONTAINER`, donde se renderizan dinámicamente las tarjetas de producto.

Laptop Asus TUF Gaming A15 - AMD Ryzen 7 7445HS

Potente laptop gaming con procesador AMD Ryzen 7 7445HS y tarjeta gráfica NVIDIA RTX. Cuenta con 16GB de memoria RAM DDR5 y 512GB SSD. Pantalla Full HD (1920x1080) diseñada para alto rendimiento en gaming y tareas exigentes.

\$17,999.00

Agregar al Carrito

Carrito

La vista del carrito incluye:

- Un título.
- Un texto introductorio.
- El contenedor `#CART-CONTAINER`, donde se genera la tabla con los productos seleccionados.
- Un botón para vaciar el carrito.

Carrito de Compras

Revisa tus productos, cantidades y el total de tu compra.

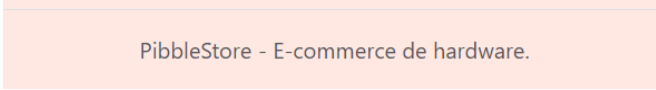
Producto	Cantidad	Precio	Subtotal	Acciones
Laptop Asus TUF Gaming A15 - AMD Ryzen 7 7445HS	1	\$17,999.00	\$17,999.00	Eliminar

Total: \$17,999.00

[Vaciar Carrito](#)

3. FOOTER

El pie de página contiene una nota simple con el nombre del proyecto y su propósito como demostración de e-commerce.



PibbleStore - E-commerce de hardware.

4. Carga de scripts y estilos

En el **HEAD** se cargan:

- Bootstrap desde CDN.
- Bootstrap Icons desde CDN.
- El archivo local `CSS/STYLES.CSS`.

Al final del documento se carga:

- `JS/APP.JS`, para que el DOM esté disponible antes de ejecutar la lógica.

Uso de Bootstrap

Bootstrap se utilizó como base para acelerar la construcción visual de la interfaz y mantener una distribución responsive sin escribir todo desde cero.

Sistema de grid

El catálogo de productos usa una estructura basada en columnas:

- `COL-12` para pantallas pequeñas.
- `COL-SM-6` para pantallas medianas.
- `COL-LG-4` para pantallas grandes.

Esto permite que:

- En móvil se muestre una tarjeta por fila.
- En tablet se muestren dos tarjetas por fila.
- En escritorio se muestren tres tarjetas por fila.

Componentes Bootstrap empleados

- `CONTAINER` para centrar el contenido.
- `ROW` y `COL-*` para la distribución responsive.
- `CARD` para las tarjetas de productos.
- `TABLE` para el carrito de compras.
- `BTN`, `BTN-PRIMARY`, `BTN-OUTLINE-PRIMARY`, `BTN-DANGER`, etc. para acciones.
- `BADGE` para mostrar la cantidad total de productos en el carrito.

- `ALERT` para indicar cuando el carrito está vacío.
- `D-NONE` para ocultar y mostrar secciones según la vista activa.
- `POSITION-RELATIVE`, `POSITION-ABSOLUTE`, `TRANSLATE-MIDDLE` para el contador del carrito.

Implementación de estilos personalizados

El archivo `CSS/STYLES.CSS` complementa Bootstrap con una identidad visual más marcada.

Variables CSS

Se usan variables dentro de `:ROOT` para centralizar colores, sombras y superficies:

- fondo principal
- superficies de tarjetas
- color de texto
- color secundario
- color de acento
- sombra de tarjetas

Esto facilita el mantenimiento porque los valores visuales se concentran en un solo lugar.

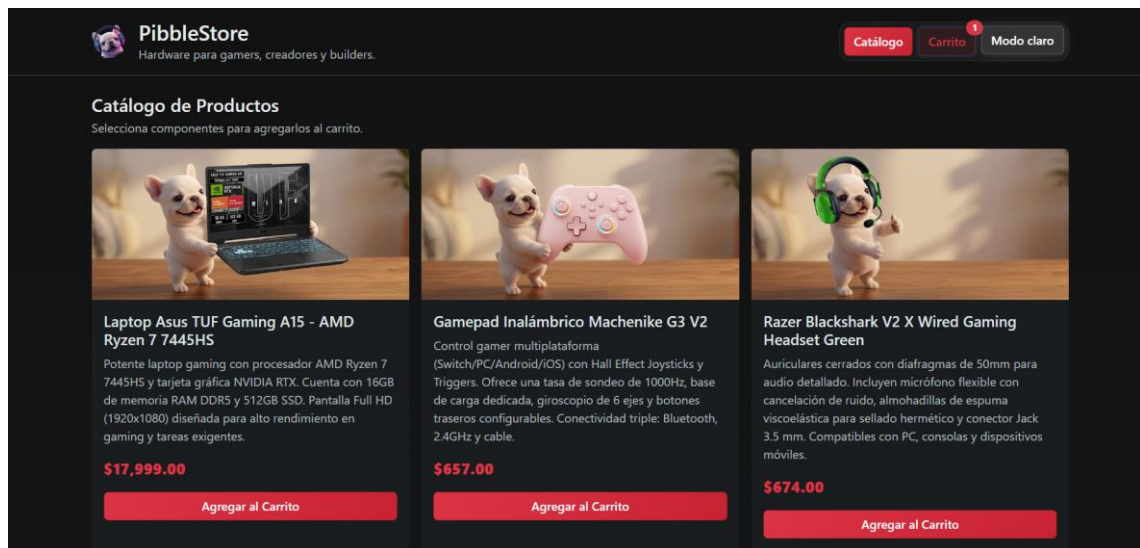
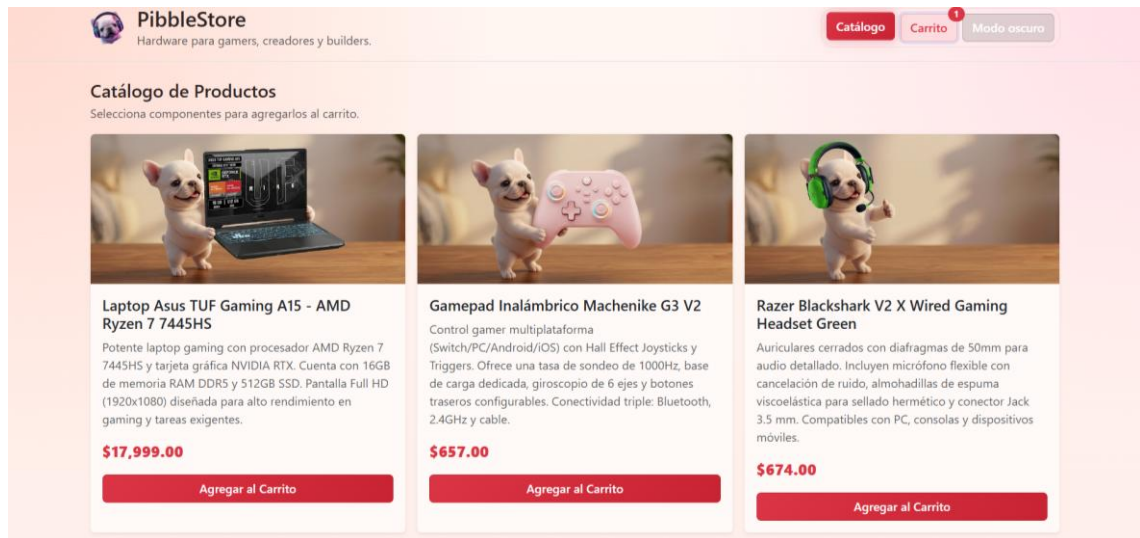
Tema oscuro y tema claro

La hoja de estilos define dos configuraciones visuales:

- Una base oscura.
- Una variante clara activada con `HTML [ DATA-BS-THEME="LIGHT" ]`.

Cuando cambia el tema:

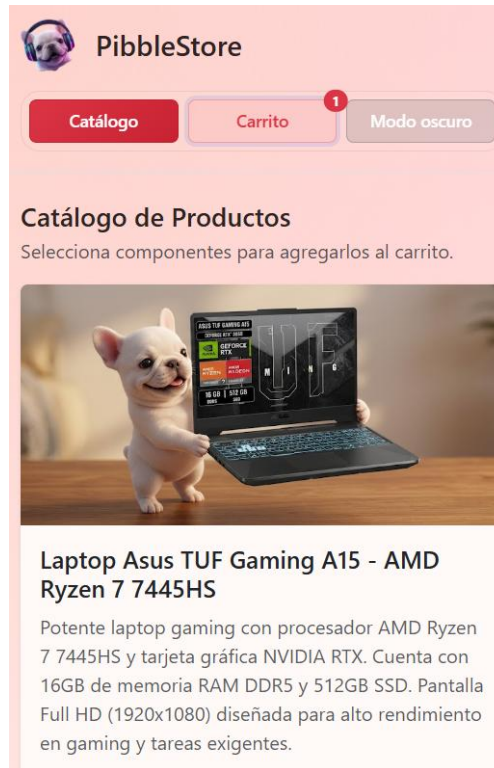
- cambian los fondos,
- cambian los colores de texto,
- cambian los bordes,
- cambian las sombras,
- cambian algunas transparencias para conservar contraste.



Responsive design

Se incluye una media query para mejorar la experiencia en móviles:

- la navegación se ajusta al ancho disponible,
- los botones se vuelven flexibles,
- el botón de vaciar carrito ocupa todo el ancho en pantallas pequeñas,
- los efectos hover se suavizan para mantener estabilidad visual.



Lógica de JavaScript

El archivo `JS/APP.JS` contiene toda la interacción dinámica del sitio.

Datos base del catálogo

El catálogo se define mediante un arreglo llamado `PRODUCTS`.

Cada objeto contiene:

- `ID`
- `NOMBRE`
- `PRECIO`
- `DESCRIPCION`
- `IMAGEN`

Los productos no se cargan desde una API externa, sino desde una lista local dentro del script. Esto simplifica el proyecto y lo hace ideal para una práctica académica o demostración front-end.

```
const products = [  
  {  
    id: 1,  
    nombre: 'Laptop Asus TUF Gaming A15 - AMD Ryzen 7 7445HS',  
    precio: 17999,  
    descripcion: 'Potente laptop gaming con procesador AMD Ryzen 7 7445HS y tarjeta gráfica NVIDIA RTX. Cuenta con 16GB de memoria RA',  
    imagen: './media/1.png',  
  },  
]
```

Funcionamiento del carrito

GETCARTFROMSTORAGE()

Esta función obtiene el carrito desde **LOCALSTORAGE**.

- Si existe información guardada, la convierte de texto a objeto con **JSON.PARSE**.
- Si no existe, devuelve un arreglo vacío.

```
function getCartFromStorage() {  
  const storedCart = localStorage.getItem(STORAGE_KEY);  
  return storedCart ? JSON.parse(storedCart) : [];  
}
```

SAVECARTTOSTORAGE(CART)

Guarda el arreglo del carrito en **LOCALSTORAGE** usando **JSON.STRINGIFY**.

Esto permite conservar:

- productos agregados,
- cantidades,
- cambios entre recargas de página.

```
function saveCartToStorage(cart) {  
  localStorage.setItem(STORAGE_KEY, JSON.stringify(cart));  
}
```

SAVETOCART(PRODUCT)

Agrega un producto al carrito con la siguiente lógica:

- si el producto ya existe, incrementa su cantidad;
- si no existe, lo inserta con **QUANTITY: 1**.

Después de actualizar el arreglo:

- se guarda en **LOCALSTORAGE**,
- se vuelve a pintar el carrito en pantalla.

```
function saveToCart(product) {  
  const cart = getCartFromStorage();  
  const existingProduct = cart.find((item) => item.id === product.id);  
  
  if (existingProduct) {  
    existingProduct.quantity += 1;  
  } else {  
    cart.push({ ...product, quantity: 1 });  
  }  
  
  saveCartToStorage(cart);  
  renderCart();  
}
```


RENDERCART()

Esta función construye la vista del carrito dinámicamente.

Primero:

- lee el carrito desde `LOCALSTORAGE`,
- actualiza el contador del carrito.

Después evalúa dos casos:

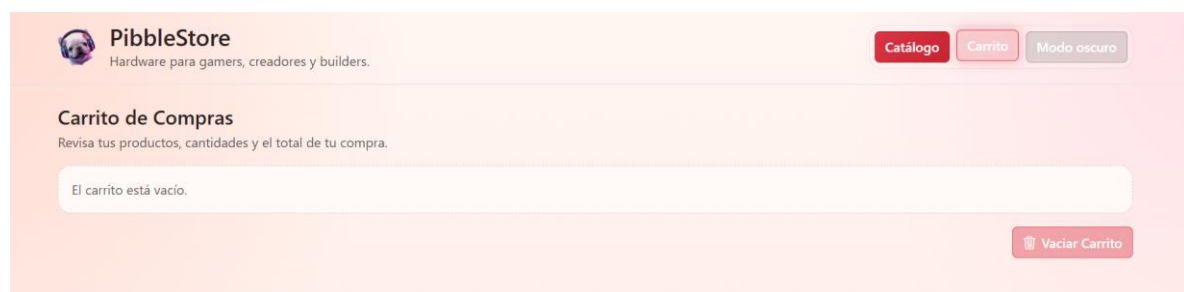
- **Carrito vacío:** muestra un mensaje de estado vacío.
- **Carrito con productos:** genera una tabla con:
 - nombre del producto,
 - cantidad,
 - precio unitario,
 - subtotal,
 - botón para eliminar.

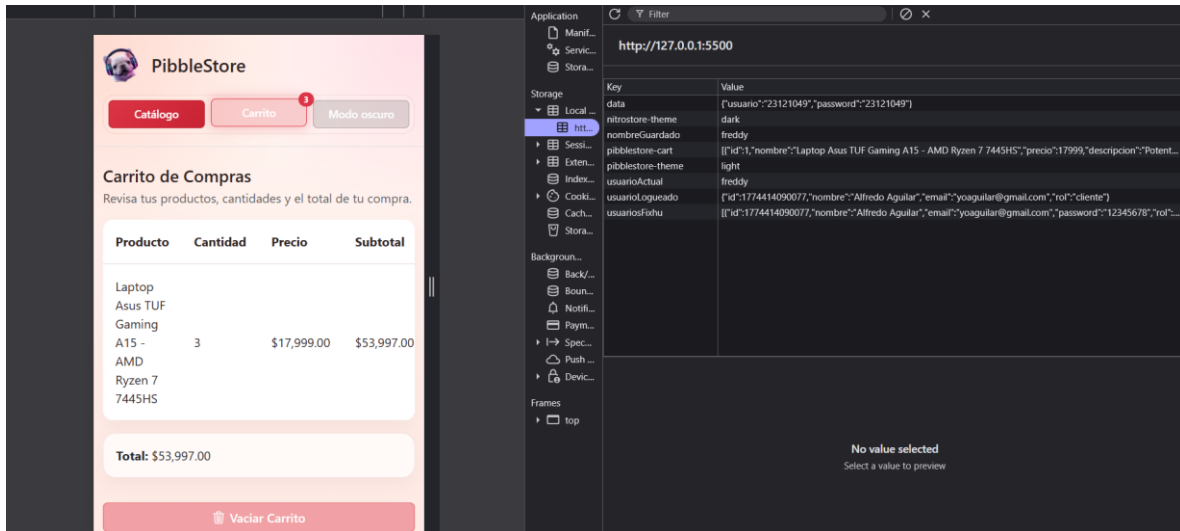
```
function renderCart() {
  const cart = getCartFromStorage();
  updateCartBadge(cart);

  if (cart.length === 0) {
    cartContainer.innerHTML = '<div class="alert alert-secondary mb-0 empty-state">El carrito está vacío.</div>';
    return;
  }

  const total = cart.reduce((accumulator, item) => {
    return accumulator + item.precio * item.quantity;
  }, 0);

  cartContainer.innerHTML = `
    <div class="table-responsive cart-table">
      <table class="table table-hover align-middle mb-0">
        <thead>
          <tr>
            <th>Producto</th>
            <th>Cantidad</th>
            <th>Precio</th>
            <th>Subtotal</th>
            <th class="text-end">Acciones</th>
          </tr>
        </thead>
        <tbody>
          ${cart
            .map(
              (item) => `
                <tr>
                  <td>${item.nombre}</td>
                  <td>${item.quantity}</td>
                  <td>${item.precio}</td>
                  <td>${item.precio * item.quantity}</td>
                  <td>
                    <button class="btn btn-sm text-danger">Eliminar</button>
                  </td>
                </tr>
              `
            )
          }
        </tbody>
      </table>
    </div>
  `;
}
```





UPDATECARTBADGE()

Actualiza el contador visible del carrito en la barra superior.

Si hay productos:

- muestra el número total de unidades,
- activa un resaltado visual en el botón del carrito.

Si no hay productos:

- oculta el contador,
- quita el estado resaltado.

```
function updateCartBadge(cart = getCartFromStorage()) {
  const totalItems = cart.reduce((accumulator, item) => accumulator + item.quantity, 0);

  if (totalItems > 0) {
    cartCountBadge.textContent = totalItems;
    cartCountBadge.classList.remove('d-none');
    cartNavButton.classList.add('has-items');
  } else {
    cartCountBadge.classList.add('d-none');
    cartNavButton.classList.remove('has-items');
  }
}
```

Eliminación de productos

El botón de eliminar producto en la tabla usa el atributo **DATA-REMOVE-ID**.

Cuando se pulsa:

- se filtra el producto correspondiente del arreglo guardado,
- se actualiza **LOCALSTORAGE**,
- se vuelve a renderizar el carrito.

Vaciar carrito

El botón **VACIAR CARRITO** elimina directamente la clave del almacenamiento:

- `LOCALSTORAGE.REMOVEITEM(STORAGE_KEY)`

Después se repinta la vista para reflejar que ya no hay productos.

Renderizado del catálogo

`RENDERPRODUCTS()`

El catálogo se genera dinámicamente dentro de `#PRODUCTS-CONTAINER`.

Cada producto se transforma en una tarjeta con:

- imagen,
- nombre,
- descripción,
- precio,
- botón para agregar al carrito.

```
const products = [
  {
    id: 1,
    nombre: 'Laptop Asus TUF Gaming A15 - AMD Ryzen 7 7445HS',
    precio: 17999,
    descripcion: 'Potente laptop gaming con procesador AMD Ryzen 7 7445HS y tarjeta gráfica NVIDIA RTX. Cuenta con 16GB de memoria RAM y almacenamiento SSD de 1TB. Ideal para gaming y productividad.',
    imagen: './media/1.png',
  },
  {
    id: 2,
    nombre: 'Gamepad Inalámbrico Machenike G3 V2',
    precio: 657,
    descripcion: 'Control gamer multiplataforma (Switch/PC/Android/iOS) con Hall Effect Joysticks y Triggers. Ofrece una tasa de sond',
    imagen: './media/2.png',
  },
]
```

Uso de localStorage

El proyecto utiliza `LOCALSTORAGE` en dos escenarios:

1. Persistencia del carrito

Clave utilizada:

- `PIBBLESTORE-CART`

Contenido:

- arreglo de productos con su cantidad.

Ventaja:

- si el usuario recarga la página, el carrito sigue ahí.

2. Persistencia del tema

Clave utilizada:

- `PIBBLESTORE - THEME`

Contenido:

- `DARK O LIGHT.`

Ventaja:

- la aplicación recuerda la preferencia visual del usuario entre sesiones.

Motivo de uso

`LOCALSTORAGE` fue una buena elección porque:

- no requiere servidor,
- no requiere base de datos,
- es suficiente para una demo front-end,
- permite persistencia simple del lado del navegador.

Flujo general de la aplicación

1. La página carga.
2. Se aplica el tema guardado antes de dibujar la interfaz.
3. JavaScript renderiza el catálogo.
4. JavaScript lee el carrito desde `LOCALSTORAGE` y lo muestra si existe.
5. El usuario puede:
 - agregar productos,
 - cambiar de vista,
 - eliminar productos,
 - vaciar el carrito,
 - alternar tema claro/oscuro.
6. Cada acción actualiza la interfaz sin recargar la página.

Conclusión

PibbleStore integra una estructura HTML clara, una capa visual personalizada sobre Bootstrap y una lógica JavaScript enfocada en interacción dinámica y persistencia local.

El resultado es una tienda demo funcional, responsive y fácil de mantener, ideal para demostrar:

- maquetación con Bootstrap,
- personalización con CSS,
- manipulación del DOM,
- uso de `LOCALSTORAGE`,

- control de vistas y estado visual.